

Auditoria de Arquitetura de IA

Mapeamento completo de uso de Gemini/OpenAI e proposta de abstracao

Data: 24 de junho de 2026

Modo: somente leitura - sem alteracao de codigo

Escopo: API Server + 9 telas mobile + proposta JADE AI Engine

1. Resumo Executivo

O projeto JADE IA usa Gemini 2.5 Flash como unico provedor de linguagem. Toda a inteligencia reside no servidor (Express API); o app mobile nunca chama um SDK de IA diretamente - apenas faz fetch para endpoints REST proprios.

Nao existe uma camada de abstracao de IA: os dois arquivos que chamam o Gemini instanciam o cliente diretamente dentro de cada handler de rota. Os prompts estao distribuidos em tres localizacoes - dentro de jade.ts, dentro de marketing.ts e embutidos em 9 telas do mobile como strings inline dentro de chamadas HTTP.

Descoberta relevante: o endpoint /api/jade/transcribe (transcricao de voz) JA usa a OpenAI (OPENAI_API_KEY) via fetch direto para Whisper-1, sem o SDK openai instalado. O projeto ja depende de dois provedores simultaneamente, mas de forma nao declarada.

2. Inventario de Provedores e SDKs

SDK de IA instalado	@google/generative-ai v0.24.1
Modelo de linguagem	gemini-2.5-flash (hardcoded em 5 lugares)
SDK OpenAI instalado	Nao - chamada via fetch nativo
GEMINI_API_KEY	Configurada (em uso)
OPENAI_API_KEY	Configurada (em uso - Whisper)
Modelo de voz	whisper-1 via OpenAI REST direto
SDK Anthropic	Nao instalado, nao referenciado
IA no mobile (SDK nativo)	Nenhum - mobile e 100% thin client

3. Arquivos que Chamam Modelos de IA

3.1 No servidor (chamadas reais)

Arquivo	Linhas	Provedores	Instancias
routes/jade.ts	1.167	Gemini + OpenAI	4x getGenerativeModel
routes/marketing.ts	71	Gemini	1 por request

Os outros 17 arquivos de rota nao fazem nenhuma chamada de IA. feedback.ts tem um endpoint /gerar que constroi e retorna um prompt como string JSON para o cliente usar - ele nunca chama o modelo diretamente.

3.2 No mobile (prompts inline em chamadas HTTP)

Arquivo mobile	Funcao	Caracteristica do prompt
(tabs)/jade.tsx	Chat principal, CRM, abordagem	messages[] com historico - uso correto
marketing.tsx	Gerar post social	Chama /jade/chat (roteamento errado)
metas.tsx	Estrategia de metas	Prompt com dados do time em runtime
planejamento.tsx	Sugestao diaria	Template com itens do planejamento
relatorios.tsx	Analise diaria/semanal	Metricas hardcoded ficticias!
roleplay.tsx	Simulacao + feedback	Injeta [SYSTEM: ...] no content
relatoriogestor.tsx	Relatorio + estrategia	2 chamadas encadeadas
feedbackjade.tsx	Feedback de vendedor	Injeta [SYSTEM: ...] no content
carteira.tsx	Analise da carteira	Prompt com resumo de clientes

Hook auxiliar hooks/useJADE.ts: wrapper que centraliza o fetch para /api/jade/chat com retry automatico (1 tentativa apos 1,5s). E uma boa base, mas usado so em alguns casos - as telas acima fazem seus proprios fetch diretos.

4. Onde os Prompts sao Montados

Os prompts estao distribuidos em 3 localizacoes distintas, sem centralizacao:

- jade.ts - JADE_SYSTEM_PROMPT: constante de ~360 linhas com 11 blocos tematicos (papel, tom, detector de compra, objecoes, BANT+SPIN, exemplos, identidade).
- jade.ts - SEGMENT_SPECIALIST: 14 blocos por segmento de mercado, concatenados dinamicamente ao system prompt em runtime com base no segmento da empresa.
- jade.ts - prompts de operacoes especificas (generateContent, sem historico): buildAnalysisOnlyPrompt (analise de lead), /prospectar (3 leads JSON), /approach (mensagem WhatsApp).
- marketing.ts - prompt concatenado: o system_context e construido no cliente mobile e enviado no corpo; o servidor apenas concatena e chama o modelo.
- 9 telas mobile - strings de prompt embutidas no corpo do fetch, construidas em runtime com dados locais. Algumas usam o hack [SYSTEM: ...] no campo content.

5. Configuracoes de Modelo por Operacao

Operacao	Endpoint	Tipo	Temp	Tokens
Chat principal	/jade/chat	Chat + historico	0.7	4000
Analise de lead	interno /chat	generateContent	0.5	500
Abordagem WhatsApp	/jade/approach	generateContent	0.85	200
Prospeccao (leads)	/jade/prospectar	generateContent	default	default
Marketing	/marketing/generate	generateContent	default	default
Transcricao	/jade/transcribe	Whisper REST	n/a	n/a

6. Funcionalidades Dependentes de IA

Total: 14 funcionalidades - 13 usando Gemini, 1 usando OpenAI (Whisper).

Funcionalidade	Endpoint / Origem	Modelo
Chat com JADE	/api/jade/chat	Gemini 2.5 Flash
Prospeccao de leads	/api/jade/prospectar	Gemini 2.5 Flash
Analise de lead real	interno em /chat	Gemini 2.5 Flash
Abordagem WhatsApp	/api/jade/approach	Gemini 2.5 Flash

Funcionalidade	Endpoint / Origem	Modelo
Transcriçao de voz	/api/jade/transcribe	OpenAI Whisper-1
Campanha de marketing	/api/marketing/generate	Gemini 2.5 Flash
Post social (mobile)	marketing.tsx	Gemini (mal roteado)
Estrategia de metas	metas.tsx	Gemini 2.5 Flash
Planejamento diario	planejamento.tsx	Gemini 2.5 Flash
Relatorio diario/semanal	relatorios.tsx	Gemini 2.5 Flash
Roleplay de vendas	roleplay.tsx	Gemini 2.5 Flash
Relatorio executivo	relatoriogestor.tsx	Gemini 2.5 Flash
Feedback de vendedor	feedbackjade.tsx	Gemini 2.5 Flash
Analise da carteira	carteira.tsx	Gemini 2.5 Flash

7. Status da OPENAI_API_KEY

A chave esta disponivel no ambiente e JA esta em uso ativo - mas de forma limitada e nao declarada.

Uso atual

Somente /api/jade/transcribe (Whisper-1), implementado com fetch nativo para api.openai.com/v1/audio/transcriptions, sem o SDK openai.

Funcionalidades prontamente migraveis para OpenAI

- /jade/chat (trocar gemini-2.5-flash por gpt-4o-mini ~ 15 linhas)
- /jade/prospectar, /jade/approach, /marketing/generate
- Analise de lead (interna ao /chat)

Principal barreira: diferenca de formato de API. Gemini usa role 'model' para respostas do assistente e systemInstruction separado; OpenAI usa role 'assistant' e messages com role 'system' no inicio do array.

8. Proposta - JADE AI Engine

Criar uma camada de servico src/lib/ai-engine.ts que abstraia todo contato com provedores de IA atraves de uma interface unificada. As rotas Express passam a ser orquestradores finos que constroem contexto e delegam ao engine.

Interface proposta

```
interface AIEngine {
  chat(req: ChatRequest): Promise<ChatResponse>
  generate(req: GenerateRequest): Promise<string>
  transcribe(req: TranscribeRequest): Promise<string>
  analyzeLeads(req: LeadAnalysisRequest): Promise<Analysis[]>
}
```

Responsabilidades centralizadas

- Selecao de modelo por operacao (chat vs. geracao vs. STT)
- Configuracao de temperatura/tokens por operacao
- Retry com backoff e timeout / circuit breaker
- Normalizacao de historico (model <-> assistant)
- Log estruturado de tokens consumidos

Repositorio de prompts (src/lib/prompts/)

- jade-system.ts -> JADE_SYSTEM_PROMPT
- segments.ts -> SEGMENT_SPECIALIST (14 blocos)

- operations.ts -> prospectar, approach, lead analysis
- marketing.ts -> prompt de campanha

Controle por variavel de ambiente

```
AI_PROVIDER=gemini|openai
AI_CHAT_MODEL=gpt-4o-mini
AI_GENERATE_MODEL=gpt-4o-mini
AI_STT_MODEL=whisper-1
```

9. Complexidade de Migracao: MEDIA

Por que nao e BAIXA

- 9 telas mobile com prompts inline - cada uma precisa ser refatorada para mover o prompt para o servidor.
- JADE_SYSTEM_PROMPT de 360 linhas augmentado dinamicamente com segmento e empresa - logica acoplada ao handler do /chat.
- Formato de historico diferente: role 'model' (Gemini) vs. 'assistant' (OpenAI); o store atualmente persiste role 'model'.
- Hack [SYSTEM: ...] em 3 arquivos precisa ser movido para systemInstruction no servidor.

Por que nao e ALTA

- Nenhum embedding, vector store, fine-tuning ou RAG
- Nenhum streaming SSE implementado (tudo e blocking await)
- O mobile ja e thin client - zero mudanca de SDK no app
- Ambas as chaves ja estao configuradas no ambiente
- A superficie da API REST permanece igual para o mobile

10. Principais Riscos

R1**ALTO**

Regressao de comportamento do chat: mudar a entrega do JADE_SYSTEM_PROMPT pode alterar sutilmente o comportamento do modelo. Comparar outputs antes/depois.

R2**ALTO**

Normalizacao de historico Gemini -> OpenAI: role 'model' persistido no store viraria 'assistant'. Historicos salvos precisam de migracao ou normalizacao.

R3**MEDIO**

Whisper e exclusivo da OpenAI: nao ha equivalente Gemini para STT de audio nativo. Migrar 100% para Gemini exigiria audio inline (API mais complexa).

R4**MEDIO**

Prompts com metricas ficticias: relatorios.tsx envia metricas hardcoded ao modelo como se fossem reais. Deve ser corrigido antes da migracao.

R5**MEDIO**

Perda de estado em memoria: sessionPendingLeads (Map em jade.ts) e perdido no restart. Ao refatorar, mover para o store persistido.

R6**BAIXO**

marketing.tsx roteia errado: chama /jade/chat em vez de /marketing/generate, ignorando o sistema de campanhas.

R7**BAIXO**

Ausencia de retry no servidor: so o mobile faz retry; falhas transientes retornam 500 imediatamente.

R8

BAIXO

Nome do modelo hardcoded em 5 lugares; atualizar o modelo exige 5 edicoes manuais.

11. Arquivos a Alterar numa Migracao Futura

Servidor (caminho critico)

Arquivo	Acao	Motivo
routes/jade.ts	Refatorar	Remover 4 instancias diretas; delegar ao engine
routes/marketing.ts	Refatorar	Remover instancia direta; delegar ao engine
lib/ai-engine.ts	Criar	Servico unificado com adapters por provedor
lib/prompts/jade-system.ts	Criar	Extrair JADE_SYSTEM_PROMPT
lib/prompts/segments.ts	Criar	Extrair SEGMENT_SPECIALIST (14 blocos)
lib/prompts/operations.ts	Criar	Extrair prospectar, approach, analysis
db/store.ts	Ajustar	Normalizar role 'model' -> 'assistant'
package.json	Add dep	openai SDK para formalizar Whisper

Mobile (centralizar prompts no servidor)

Arquivo	Acao	Endpoint novo sugerido
marketing.tsx	Redirecionar	/api/marketing/generate
metas.tsx	Mover prompt	/api/metas/estrategia
planejamento.tsx	Mover prompt	/api/planejamento/sugestao
relatorios.tsx	Mover prompt + dados	/api/relatorios/analise
roleplay.tsx	Mover system prompt	/api/roleplay/chat + /feedback
relatoriogestor.tsx	Mover prompts	/api/dashboard/relatorio + /estrategia
feedbackjade.tsx	Mover system prompt	/api/feedback/vendedor
carteira.tsx	Mover prompt	/api/carteira/analise

12. Conclusao

O projeto JADE IA tem uma arquitetura de IA funcional mas monolitica - toda a inteligencia esta concentrada em jade.ts sem separacao de responsabilidades. Funciona bem no estado atual, mas e dificil de manter, testar e migrar de provedor.

Os tres achados mais criticos:

- Prompts distribuidos em 12 locais (1 servidor + 11 mobile) tornam auditoria e atualizacao de comportamento arriscados.
- OpenAI ja esta em uso (Whisper) sem estar declarada como dependencia oficial - o sistema ja depende de dois provedores.
- Metricas ficticias hardcoded em relatorios.tsx sao enviadas ao modelo como se fossem dados reais - precisa ser corrigido independentemente de qualquer migracao.

A proposta JADE AI Engine e viavel com complexidade MEDIA: estimativa de 4-6 dias para a camada de servico no servidor e 3-5 dias para migrar os prompts das telas mobile - totalizando ~2 semanas de sprint tecnico sem regressao de produto.